

O estado atual mostra uma solução já distribuída em produção com frontend em Cloudflare Pages, backend/middleware/admin em Hetzner via Docker, banco PostgreSQL com pgvector, túnel Cloudflare e integração Protheus por AdvPL REST, o que prova viabilidade técnica e uma arquitetura coerente para SaaS híbrido.[^2][^1]

O principal gap para torná-la comercial não é “fazer funcionar”, e sim transformar a base atual em produto multiempresa governável, seguro, auditável, replicável e com onboarding padronizado.[^1][^2]

Diagnóstico

Pelos documentos, já existem elementos importantes de produto: separação de serviços, painel administrativo, suporte a LLM remoto e local, RAG com R2, banco vetorial e noções de tenants/empresas no backend.[^2][^1]

Ao mesmo tempo, aparecem sinais de maturidade ainda parcial para comercialização: gestão de segredos concentrada em `.env``, credenciais administrativas simples, dependência de deploy manual no admin, acoplamento operacional à VPS única e ausência, nos documentos, de evidências claras sobre isolamento forte por tenant, billing, trilha de auditoria, observabilidade centralizada, RBAC e onboarding self-service.[^1][^2]

Hipóteses e evidências

****Evidências disponíveis:**** a solução usa microserviços containerizados, backend FastAPI, middleware Node/Express, painel React/Vite, Cloudflare Tunnel, PostgreSQL + pgvector, R2 e integração AdvPL REST para Protheus, com fluxo já implantado e comandos de operação definidos.[^2][^1]

****Hipótese funcional:**** o produto já atende bem um cenário single-company ou poucos clientes geridos manualmente, mas ainda depende de operação assistida para implantação, configuração e suporte.[^1][^2]

****Hipótese técnica:**** existe uma base inicial para multiempresa porque o backend menciona tenants e empresas e o middleware consome `x-tenant-id``, porém falta validar se isso está refletido ponta a ponta em autenticação, autorização, segregação de dados, storage RAG, logs, métricas e integrações por cliente.[^1]

Impactos

Para virar comercial, os maiores impactos estarão em cadastro e governança de ****tenant/empresa****, credenciais por cliente, bases RAG segregadas, parametrização por ambiente Protheus e políticas de acesso no painel administrativo.[^2][^1]

Também haverá impacto técnico em banco, storage, observabilidade, deploy e suporte, porque cada cliente precisará de isolamento mínimo de dados e rastreabilidade operacional para reduzir risco contratual, fiscal e reputacional.[^2][^1]

Recomendação

Eu recomendo tratar a demanda como ****novo desenvolvimento + melhoria de processo + preparação SaaS multiempresa****.[^1][^2]

1) Prioridade imediata: prontidão comercial

- Formalizar o modelo de tenancy: tenant, empresa, usuário, ambiente Protheus, conectores e base RAG por cliente.[^1]
- Definir a estratégia de isolamento: banco compartilhado com `tenant\_id` e políticas rígidas, ou isolamento lógico/físico por cliente para os componentes mais sensíveis.[^2][^1]
- Remover credenciais frágeis/default do fluxo operacional e migrar segredos para cofre ou gestão segura de secrets; os documentos mostram `ADMIN\_USER/ADMIN\_PASSWORD` e `JWT\_SECRET` em `.env`, o que é insuficiente para escala comercial.[^2][^1]
- Criar RBAC no admin: superadmin Elite, admin do cliente, operador, auditor.[^1]

2) Multiempresa de verdade

- Garantir que todo request carregue contexto de tenant validado no backend, middleware, consultas RAG, upload de documentos e chamadas Protheus.[^1]
- Segregar knowledge base por tenant no R2 e nos metadados vetoriais, com purge e reindex independentes.[^2]
- Configurar múltiplos endpoints/credenciais Protheus por empresa/filial/ambiente, porque cada cliente poderá ter VPN, appserver, REST e regras próprias.[^2]

3) Segurança e compliance operacional

- Implementar auditoria: quem perguntou, qual tenant, qual fonte respondeu, se houve chamada transacional ao Protheus e qual retorno.[^1]
- Adotar logs estruturados, monitoramento e alertas; hoje há evidência de comandos de sobrevivência, mas não de observabilidade madura.[^2]
- Padronizar backup/restore de PostgreSQL, R2 e configurações do painel, com RTO/RPO definidos.[^1][^2]

4) Produto comercial

- Criar onboarding guiado: cadastro do cliente, validação de conexão Protheus, upload da base documental, escolha do motor LLM, teste de saúde e ativação.[^2][^1]
- Definir pacotes comerciais: assistente RAG, assistente com dados online, assistente com ações transacionais, BI/Power BI, IA local opcional.[^1][^2]
- Produzir documentação de implantação, critérios de aceite, SLA e matriz de responsabilidades entre Elite, cliente e infraestrutura.[^2][^1]

Backlog inicial

- ****Tenant core:**** entidades tenant, company, environment, connector, knowledge_base, user_role.[^1]
- ****Auth/RBAC:**** login forte, rotação de senha, perfis e escopo por tenant.[^1]

- **RAG multiempresa:** bucket/prefix por tenant, metadados por coleção, reindex por cliente.[^2]
- **Conectividade Protheus:** cadastro de endpoints, headers, chaves, healthcheck e versionamento por cliente.[^2][^1]
- **Auditoria:** log funcional e técnico de perguntas, respostas, fontes e ações.[^1]
- **Operação:** CI/CD também para admin-frontend, hoje dependente de rebuild manual na VPS.[^2]
- **Observabilidade:** dashboard de uptime, latência, consumo por tenant, falhas de integração e custo por modelo LLM.[^1][^2]
- **Comercialização:** telas de onboarding, trial, limites por plano e preparação para billing.[^1]

Riscos

- **Risco de vazamento entre clientes** se o `tenant\_id` não estiver sendo imposto de forma consistente em todas as camadas.[^1]
- **Risco operacional** pela dependência de procedimentos manuais de deploy e administração em uma VPS central.[^2]
- **Risco de segurança** por secrets e credenciais administrativas tratados de forma simples nos documentos.[^2][^1]
- **Risco contratual/comercial** se não houver auditoria e trilha de execução para respostas baseadas em ERP e ações automáticas.[^1]

Validações pendentes

Antes de concluir o desenho comercial, eu validaria estes pontos:

- O banco já possui tabela e chave de segregação por tenant em todas as entidades críticas?[^1]
- O RAG já separa embeddings e documentos por cliente ou ainda compartilha coleção?[^2][^1]

- O middleware já valida autorização do tenant ou apenas recebe cabeçalhos?[^1]
- O admin já possui perfis e escopos ou é um painel central único?[^1]
- Existe trilha de auditoria de ação transacional no Protheus?[^2][^1]
- O frontend do chat já suporta troca de empresa/ambiente/filial por contexto?[^2]

Na pasta C:\projeto\copilotprotheus\etc te entreguei um pacote inicial de ****scripts prontos para execução na VPS****, alinhado com a arquitetura descrita nos documentos: frontend do chat no Cloudflare Pages, admin/backend/middleware/db na Hetzner via Docker, PostgreSQL com pgvector, Cloudflare Tunnel e Ollama rodando no host Linux.[^1][^2]

O que foi entregue

O pacote inclui scripts para deploy completo, deploy só do admin, healthcheck da stack, backup do PostgreSQL/pgvector e templates de `.env``, além de um ``docker-compose.override`` inicial para endurecimento operacional multiempresa.[^2][^1]

Esses artefatos seguem diretamente o modelo operacional documentado, inclusive o rebuild manual do painel admin na VPS e a comunicação do backend com o Ollama em ``host.docker.internal:11434``.[^2]

Como usar

Na VPS, coloque os arquivos na raiz do repositório, ajuste permissões com ``chmod +x *.sh`` e execute conforme a necessidade operacional, por exemplo deploy completo, rebuild do admin ou backup do banco.[^1][^2]

Os scripts assumem que existe um ``docker-compose.yml`` na raiz, que os serviços usam nomes compatíveis com ``backend``, ``middleware``, ``admin-frontend`` e ``db``, e que o frontend do chat continua sendo publicado automaticamente pelo Cloudflare Pages a partir do GitHub.[^1][^2]

Hipóteses e cuidados

****Evidência disponível:**** os documentos confirmam a topologia e os comandos-base de operação, mas não trazem o conteúdo real do `docker-compose.yml`, nem os nomes exatos finais dos serviços e endpoints de healthcheck.[^2][^1]

****Hipótese técnica:**** os scripts vão funcionar com pouco ajuste, porém você provavelmente precisará validar nomes dos serviços, paths do projeto, credenciais, branch e endpoints expostos pelo backend antes de rodar em produção.[^1][^2]

Impactos

Esses scripts aceleram operação e padronizam deploy, backup e diagnóstico, reduzindo dependência de execução manual ad hoc na Hetzner.[^2]

Para cenário comercial multiempresa, ainda faltam scripts de provisionamento por tenant, rotação de secrets, restore testado, migração de banco e talvez automação de CI/CD do admin, porque hoje o processo documentado ainda depende de SSH e rebuild manual.[^1][^2]

Próximos passos

1. Validar os nomes reais do `docker-compose.yml` e ajustar os scripts ao repositório.[^2][^1]
2. Trocar todos os valores default dos templates de ambiente por secrets fortes antes de qualquer uso produtivo.[^1][^2]
3. Executar primeiro `healthcheck\_stack.sh` em homologação e depois `deploy\_full.sh` no ambiente alvo.[^2]
4. Na sequência, eu posso te entregar uma ****versão 2**** com scripts de provisionamento multiempresa, restore, rotação de backup, systemd para Ollama e checklist de go-live comercial.